

Android #10 Library dan Dependency Injection

Table of Contents

<i>Android #10 Library dan Dependency Injection</i>	1
Table of Contents	1
Library	1
Manfaat Library	1
Dependency Injection (DI)	2
Library yang Digunakan dalam Praktik	2
Dependency Injection dengan Hilt	2
Perbandingan Pengembangan Aplikasi Tanpa Hilt (DI) dan dengan Hilt (DI)	11
Referensi	11
Tugas Pendalaman Materi	11

Library

Dalam pengembangan perangkat lunak, Library adalah kumpulan kode yang telah ditulis sebelumnya yang dapat digunakan oleh pengembang untuk mengoptimalkan tugas-tugas umum. Daripada menulis kode dari nol untuk fungsi yang kompleks (seperti mengunduh gambar atau mengelola koneksi database), pengembang cukup memanggil fungsi yang sudah disediakan oleh library tersebut.

Manfaat Library

- Efisiensi Waktu: Mempercepat proses pengembangan.
- Standarisasi: Menggunakan solusi yang sudah teruji oleh komunitas global.
- Fokus pada Logika Bisnis: Pengembang bisa fokus pada fitur unik aplikasi mereka, bukan pada masalah infrastruktur dasar.

Dependency Injection (DI)

Dependency Injection adalah sebuah pola desain (design pattern) di mana sebuah objek tidak lagi bertanggung jawab untuk membuat objek lain yang dibutuhkannya (dependensi) secara mandiri. Sebaliknya, dependensi tersebut "disuntikkan" ke dalam objek tersebut oleh pihak luar (dalam hal ini, Hilt).

Tanpa DI, class-class dalam aplikasi akan saling terikat sangat kuat (tightly coupled). Jika ada perubahan pada satu class, class lain yang bergantung padanya harus ikut diubah manual. Dengan DI, kode menjadi lebih modular, mudah diuji (testable), dan mudah dipelihara (maintainable).

Library yang Digunakan dalam Praktik

Retrofit & GSON

Retrofit : Bertindak sebagai jembatan komunikasi antara aplikasi Android dengan server (API). Retrofit mengubah interface Java/Kotlin menjadi panggilan jaringan yang nyata.

GSON : Berperan sebagai penerjemah. Data dari internet biasanya berbentuk format JSON (teks). GSON secara otomatis mengubah teks tersebut menjadi objek Data Class Kotlin (Post) sehingga bisa langsung digunakan dalam kode.

Glide

Peran : Library manajemen media yang kuat untuk Android.

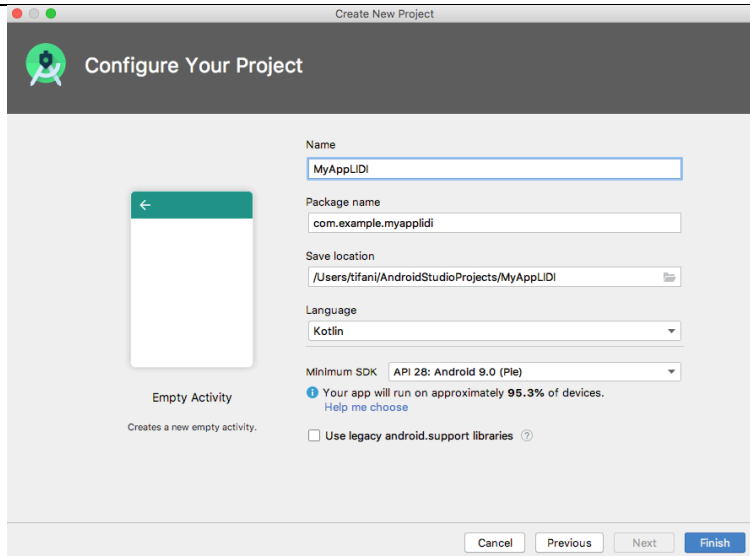
Fungsi Utama : Mengunduh gambar dari URL secara asinkron, mengelola memori agar aplikasi tidak force close karena foto terlalu berat, dan menyimpan gambar di cache agar saat dibuka kembali tidak perlu mengunduh ulang.

Dependency Injection dengan Hilt

Hilt adalah library Dependency Injection standar dari Google yang dibangun di atas Dagger. Hilt menyederhanakan penggunaan DI dengan menyediakan cara yang lebih mudah untuk mendefinisikan di mana objek harus dibuat dan di mana objek tersebut harus dikirim.

Penerapan Library Pihak Ketiga dan Dependency Injection pada Android Kotlin

1. Buat projec android seperti biasa dengan Buka Android Studio -> Start a New Android Studio Project -> Next -> Empty Activity -> Next



Finish.

2. Konfigurasi Gradle yang build.gradle (Project: AppNameKalian) dan tambahkan classpath berikut (*highlight kuning*):

```
dependencies {
    classpath "com.android.tools.build:gradle:4.0.1"
    classpath "org.jetbrains.kotlin:kotlin-gradle-plugin:$kotlin_version"

    classpath 'com.google.dagger:hilt-android-gradle-plugin:2.28-alpha'

    // NOTE: Do not place your application dependencies here; they belong
    // in the individual module build.gradle files
}
```

3. Tambahkan baris kode berikut pada Gradle yang build.gradle (Module: app) berikut (*highlight kuning*):

```
apply plugin: 'com.android.application'
apply plugin: 'kotlin-android'
apply plugin: 'kotlin-android-extensions'
// Apply baris berikut #10
apply plugin: 'kotlin-kapt'
apply plugin: 'dagger.hilt.android.plugin'
```

```
android {
    compileSdkVersion 29
    buildToolsVersion "30.0.3"

    // Tambahkan baris berikut #10
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }

    kotlinOptions {
        jvmTarget = '1.8'
    }

    defaultConfig {
```

```

        applicationId "com.example.lidiapp"
        minSdkVersion 28
        targetSdkVersion 29
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-
optimize.txt'), 'proguard-rules.pro'
        }
    }
}

```

```

dependencies {
    implementation fileTree(dir: "libs", include: ["*.jar"])

    implementation "org.jetbrains.kotlin:kotlin-stdlib:$kotlin_version"

    implementation 'androidx.core:core-ktx:1.3.2'
    implementation 'androidx.appcompat:appcompat:1.2.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.0.4'
    implementation 'com.google.android.material:material:1.2.1'
    implementation 'androidx.legacy:legacy-support-v4:1.0.0'

    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'androidx.test.ext:junit:1.1.2'
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.3.0'
    compileOnly 'com.google.android.wearable:wearable:2.5.0'

    implementation 'androidx.appcompat:appcompat:1.2.0'
    implementation 'androidx.fragment:fragment-ktx:1.2.5'
    implementation 'com.google.android.material:material:1.2.1'
    implementation 'androidx.constraintlayout:constraintlayout:2.0.4'

    implementation 'com.squareup.retrofit2:retrofit:2.6.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.6.0'

    // 1. GLIDE (Library) #10
    implementation 'com.github.bumptech.glide:glide:4.12.0'
    kapt 'com.github.bumptech.glide:compiler:4.12.0'

    // 2. HILT (Dependency Injection) #10
    implementation "com.google.dagger:hilt-android:2.28-alpha"
    kapt "com.google.dagger:hilt-android-compiler:2.28-alpha"

    // 3. Lifecycle (#11)
    implementation 'androidx.hilt:hilt-lifecycle-viewmodel:1.0.0-alpha01'
    kapt 'androidx.hilt:hilt-compiler:1.0.0-alpha01'
}

```

**dependencies yang lain dapat disesuaikan dengan project anda sebelumnya*

4. Tambahkan izin internet pada AndroidManifest.xml seperti berikut:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.lidiapp">

```

```

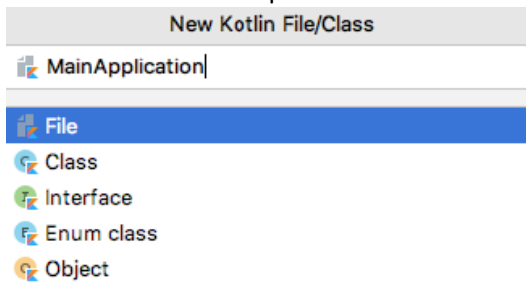
<!-- IZIN AKSES INTERNET #10-->
<uses-permission android:name="android.permission.INTERNET" />
<application
    android:name=".MainApplication"
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportRtl="true"
    android:theme="@style/AppTheme">
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>

</manifest>

```

5. Buat file MainApplication untuk mengaktifkan Hilt dengan cara klik Kanan package kemudian New dan pilih Kotlin File/Class seperti berikut:



Kemudian tambahkan kode berikut:

```

package com.example.lidiapp

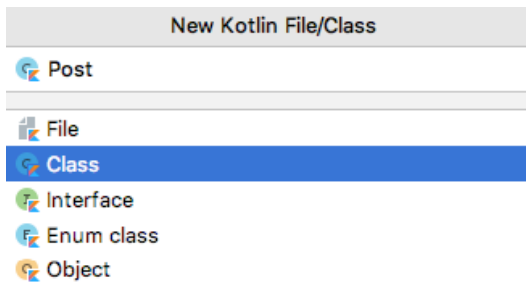
import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class MainApplication : Application()

```

6. Definisikan data model dengan cara klik Kanan package kemudian New dan pilih Kotlin File/Class seperti berikut:

Data Class:



Kemudian tambahkan kode berikut:

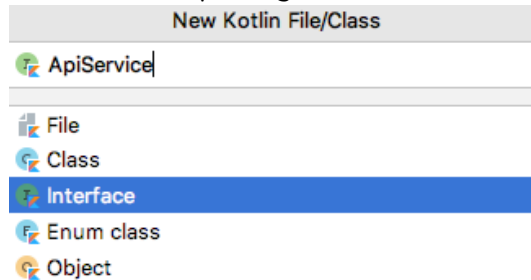
```

package com.example.lidiapp

```

```
data class Post(
    val id : Int,
    val title: String
)
```

7. Selain membutuhkan data class, kita juga membutuhkan interface, tambahkan dengan cara klik Kanan package kemudian New dan pilih Kotlin File/Class seperti berikut:



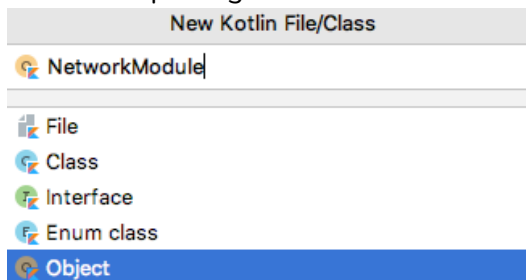
Kemudian tambahkan kode berikut:

```
package com.example.lidiapp

import retrofit2.Call
import retrofit2.http.GET

interface ApiService {
    @GET("posts")
    fun getPosts(): Call<List<Post>>
}
```

8. Tambahkan kembali interface setup hilt dengan nama NetworkModule.kt dengan cara klik Kanan package kemudian New dan pilih Kotlin File/Class seperti berikut:



Kemudian tambahkan kode berikut:

```
package com.example.lidiapp

import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.components.ApplicationComponent
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

@Module
@InstallIn(ApplicationComponent::class)
object NetworkModule {
    @Provides
    fun provideRetrofit(): Retrofit = Retrofit.Builder()
        .baseUrl("https://jsonplaceholder.typicode.com/")
        .addConverterFactory(GsonConverterFactory.create())
        .build()

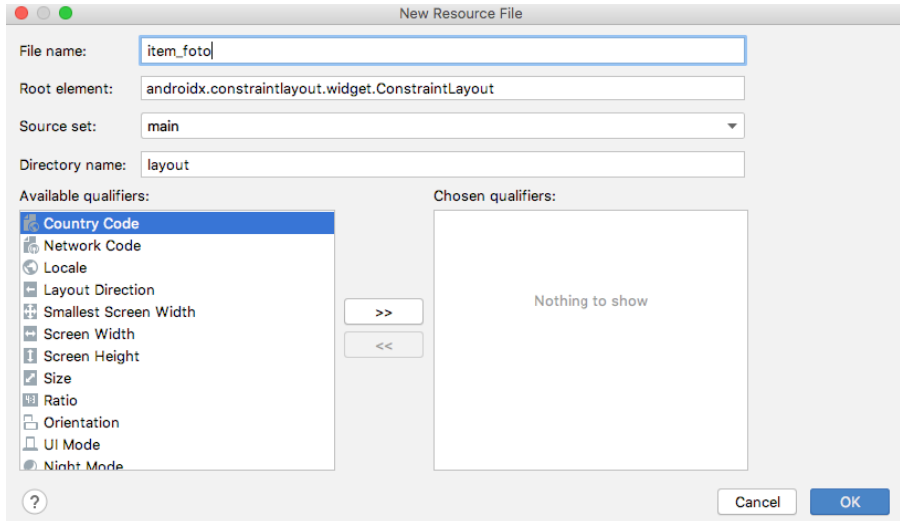
    @Provides
```

```

fun provideApiService(retrofit: Retrofit): ApiService =
    retrofit.create(ApiService::class.java)
}

```

9. Tambahkan file item_foto.xml untuk menyisipkan foto dan judul menggunakan cardview dengan cara klik kanan pada res/layout pilih New Resource File seperti berikut:



Kemudian tambahkan kode berikut:

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_margin="4dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:background="?android:attr/selectableItemBackground">

        <ImageView
            android:id="@+id/ivFoto"
            android:layout_width="match_parent"
            android:layout_height="100dp"
            android:scaleType="centerCrop"
            android:contentDescription="@string/app_name"
            android:background="@android:color/darker_gray" />

        <TextView
            android:id="@+id/tvTitle"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="4dp"
            android:maxLines="2"
            android:ellipsize="end"
            android:text="Loading Title..."
            android:textColor="@android:color/black"

```

```

        android:textSize="10sp"
        android:textStyle="bold" />

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

10. Perbarui activity_main.xml menjadi seperti berikut:

```

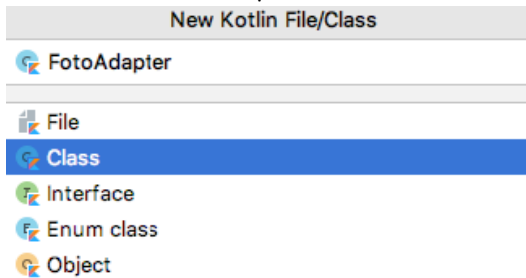
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvFoto"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:padding="2dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:listitem="@layout/item_foto" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

11. Tambahkan file FotoAdapter.kt untuk mengambil foto berdasarkan id dan text view akan menampilkan judul, caranya klik kanan pada package kemudian New dan pilih Kotlin File/Class seperti berikut:



Kemudian tambahkan kode berikut:

```

package com.example.lidiapp

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.bumptech.glide.Glide

class FotoAdapter(private val list: List<Post>) :
    RecyclerView.Adapter<FotoAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val ivFoto: ImageView = view.findViewById(R.id.ivFoto)
    }
}

```



```

        val tvTitile: TextView = view.findViewById(R.id.tvTitle)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
FotoAdapter.ViewHolder {
        val view =
LayoutInflater.from(parent.context).inflate(R.layout.item_foto, parent,
false)
        return ViewHolder(view)
    }

    override fun getItemCount(): Int {
        return list.size
    }

    override fun onBindViewHolder(holder: FotoAdapter.ViewHolder, position:
Int) {
        val item = list[position]
        holder.tvTitile.text = item.title

        val imageUrl = "https://picsum.photos/seed/${item.id}/200"

        Glide.with(holder.itemView.context)
            .load(imageUrl)
            .placeholder(android.R.drawable.ic_menu_gallery)
            .into(holder.ivFoto)
    }
}

```

12. Perbarui MainActivity menjadi seperti berikut:

```

package com.example.lidiapp

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Toast
import androidx.recyclerview.widget.GridLayoutManager
import androidx.recyclerview.widget.RecyclerView
import dagger.hilt.android.AndroidEntryPoint
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response
import java.util.*
import javax.inject.Inject

@AndroidEntryPoint
class MainActivity : AppCompatActivity() {

    @Inject
    lateinit var apiService: ApiService //Inject otomatis ke Hilt tanpa
perlu Retrofit.Builder() disini

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rvFoto: RecyclerView = findViewById(R.id.rvFoto)
        rvFoto.layoutManager = GridLayoutManager(this, 4)

        ambilDataFoto(rvFoto)
    }
}

```

```

private fun ambilDataFoto(recyclerView: RecyclerView) {
    apiService.getPosts().enqueue(object : Callback<List<Post>> {
        override fun onFailure(call: Call<List<Post>>, t: Throwable) {
            Toast.makeText(this@MainActivity, "Gagal: ${t.message}",
            Toast.LENGTH_SHORT).show()
        }

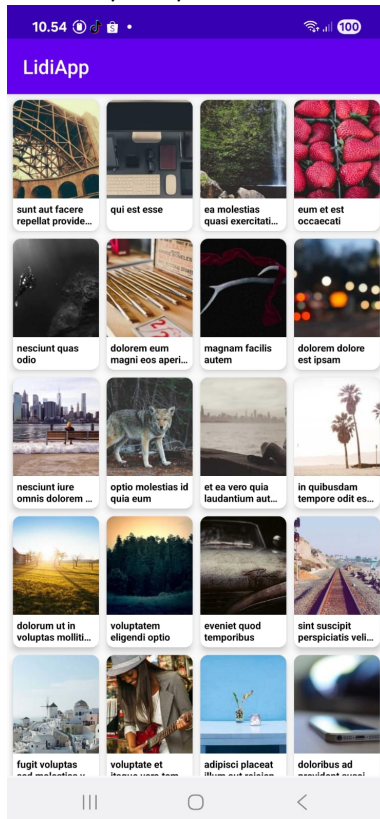
        override fun onResponse(call: Call<List<Post>>, response:
        Response<List<Post>>) {
            if(response.isSuccessful) {
                val listData = response.body()?.take(20) ?: emptyList()
                //ambil 20 data

                //pasang data ke adapter
                val adapter = FotoAdapter(listData)
                recyclerView.adapter = adapter
            }
        }
    });
}

```

13. Anda bisa melakukan Rebuild Project dengan Build -> Clean Project, jika sudah selesai lakukan Build->Rebuild Project untuk memastikan bahwa dependency sudah di build ulang.

14. Hasilnya seperti berikut ini:



Perbandingan Pengembangan Aplikasi Tanpa Hilt (DI) dan dengan Hilt (DI)

Fitur	Tanpa Hilt (DI)	Dengan Hilt (DI)
Instansiasi Retrofit	...	...
...	...	...
...	...	...
...	...	...
..	...	...

Referensi

<https://developer.android.com/>

Tugas Pendalaman Materi

Lakukan perubahan konsep pada tugas studi kasus anda sebelumnya (menampilkan data API tanpa DI) dengan konsep yang dipelajari saat ini (menampilkan data API dengan DI). Selanjutnya lakukan pengamatan terhadap perbandingan keduanya berdasarkan tabel di atas!